YACHAY TECH

School of Mathematical and Computational Sciences

Operating Systems II

MidTerm Exam Report

Author: Mateo Alejandro Chalacán Sandoval
Professor: Francisco Hidrobo

March 17, 2026

1 Introduction

This document presents the resolution of the partial exam (MidTerm), which consists of modifying the `sysinfo_chardev` kernel module. The main objective is to provide the `/dev/sysinfo` character device with the ability to receive instructions from user-space through write operations (`write`). Depending on the sent command (`all`, `kernel`, `cpu`, `time`), the device filters and returns only the requested information when a read operation (`read`) is performed.

2 Environment Description

- **Host System (Cross-Compilation):** Ubuntu-based distribution (x86_64 Architecture).
- **Target System:** Raspberry Pi (ARM64 Architecture).
- **Kernel Version on Target:** `6.12.62+rpt-rpi-v8`

3 Justification: Character Device vs. Block Device

Question: Explain why a character device driver is appropriate for implementing `/dev/sysinfo` instead of a block device.

Answer: A character device is the most suitable option for `/dev/sysinfo` due to the type of data it handles. These devices process information as a continuous and sequential stream of bytes (unbuffered). This is ideal for generating and sending short text strings directly to the user instantaneously, as well as for receiving simple commands.

Conversely, block devices are designed for mass storage. They work by accessing fixed-size data blocks (e.g., 512 bytes or 4 KB), rely heavily on the kernel cache, and are designed to support complete file systems. Therefore, using all that heavy infrastructure just to read and write a few bytes of text would be inefficient, unnecessarily complex, and a design flaw.

4 Implementation and Testing (Linux Commands)

The commands executed on the Raspberry Pi to interact with the modified module are detailed below.

4.1 Loading, Node Creation, and Permissions

To deploy the module on the system and enable interaction, the following was executed:

```
# Load the module
sudo insmod ./sysinfo_chardev.ko

# Get the 'major number' assigned by the kernel (e.g., 236)
dmesg | tail -n 2

# Create the device node
sudo mknod /dev/sysinfo c 236 0

# Set permissions to allow read/write without sudo
sudo chmod 666 /dev/sysinfo
```

4.2 Command Testing and Results

To verify the correct operation of the information filtering, the different commands defined in the exam requirements were sent:

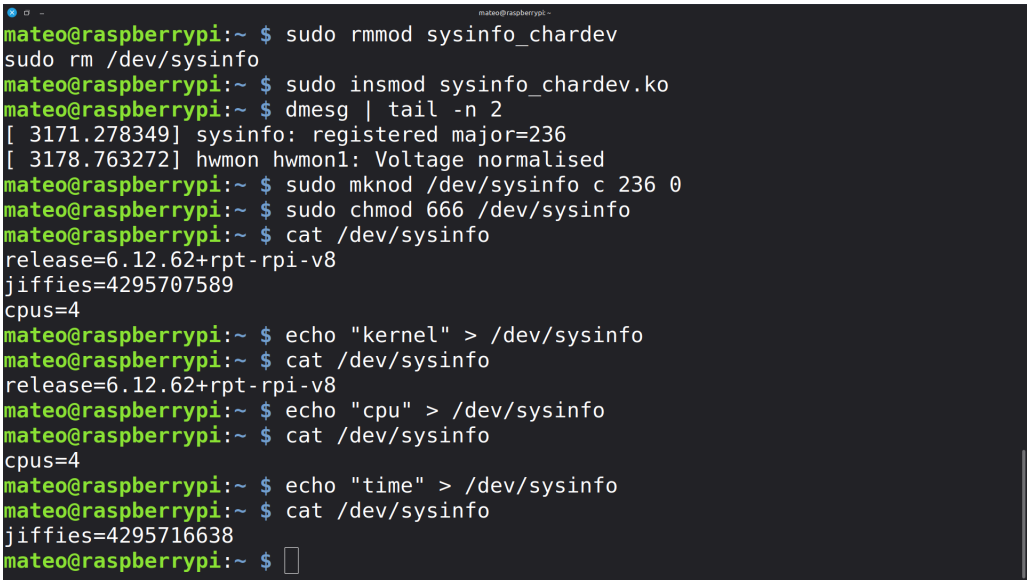
```
cat /dev/sysinfo
echo "kernel" > /dev/sysinfo
cat /dev/sysinfo
echo "cpu" > /dev/sysinfo
cat /dev/sysinfo
echo "time" > /dev/sysinfo
cat /dev/sysinfo
```

Figure 1 shows the screenshot of the terminal on the Raspberry Pi, demonstrating how the device modifies its output based on the received `echo` commands.

5 Github Link

The codes were charged to a github repository as required in the description of the activity:

https://github.com/MateoZolfot/OS-2/tree/main/MidTerm



```
mateo@raspberrypi:~ $ sudo rmmod sysinfo_chardev
sudo rm /dev/sysinfo
mateo@raspberrypi:~ $ sudo insmod sysinfo_chardev.ko
mateo@raspberrypi:~ $ dmesg | tail -n 2
[ 3171.278349] sysinfo: registered major=236
[ 3178.763272] hwmon hwmon1: Voltage normalised
mateo@raspberrypi:~ $ sudo mknod /dev/sysinfo c 236 0
mateo@raspberrypi:~ $ sudo chmod 666 /dev/sysinfo
mateo@raspberrypi:~ $ cat /dev/sysinfo
release=6.12.62+rpt-rpi-v8
jiffies=4295707589
cpus=4
mateo@raspberrypi:~ $ echo "kernel" > /dev/sysinfo
mateo@raspberrypi:~ $ cat /dev/sysinfo
release=6.12.62+rpt-rpi-v8
mateo@raspberrypi:~ $ echo "cpu" > /dev/sysinfo
mateo@raspberrypi:~ $ cat /dev/sysinfo
cpus=4
mateo@raspberrypi:~ $ echo "time" > /dev/sysinfo
mateo@raspberrypi:~ $ cat /dev/sysinfo
jiffies=4295716638
mateo@raspberrypi:~ $
```

Figure 1: Successful execution and command filtering in `/dev/sysinfo`